QASM simulator time issue

https://github.com/qiskit-community/qiskit-aqua/issues/1405

ROW 37

QASM simulator time issue #1405

Closed

MariaSapova opened on Nov 4, 2020

Information

- **Qiskit Aqua version**: 0.19.0 - 0.23.0
- **Python version**: 3.6
- **Operating system**: Ubuntu

What is the current behavior?

Function evaluation takes inadequate time in recent versions of Qiskit.
I checked Qiskit performance for VQE-UCCSD simulation of LiH molecule. Full simulation with 12 qubits can not be performed starting from version 0.19.4 (at least first function evaluation takes more than half an hour!). The same calculation with 0.19.3 version takes 5.20 sec for the first evaluation.
I also checked 6 qubit simulation with reduced active space for LiH molecule. Full optimization took 2 sec (user time) using qiskit 0.19.3 and 1 min 20 sec using qiskit 0.23.0.
This makes simulation of large molecules impossible with the latest versions.

Steps to reproduce the problem

Run the gist.

manoelmarques on Nov 5, 2020 Collaborator

I made a small change to your script and ran it using qiskit 0.23.0, python 3.6.12 and Ubuntu 20.10
It is much faster if you use Aer and qasm_simulator with include_custom=True in VQE which gives the same result as statevector simulator (less than 7 minutes):

```
if __name__ == "__main__":
    atom = 'Li 0.0 0.0 0.0; H 0.0 0.0 1.547'
    freeze_list = [0]
    remove_list = [-3, -2]

    ferOp, ferOp_red, energy_shift, num_spin_orbitals, num_particles, repulsion_energy, molecule = molecule_ir
    qubitOp = ferOp.mapping(map_type = 'jordan_wigner', threshold = 10**(-8))
    qubitOp_red = ferOp_red.mapping(map_type = 'jordan_wigner', threshold = 10**(-8))

    initial_state, var_form = uccsd_full('jordan_wigner', qubitOp.num_qubits, molecule.num_alpha + molecule.nu
    initial_state_red, var_form_red = uccsd_full('jordan_wigner', qubitOp_red.num_qubits, num_particles)

    optimizer = SLSQP(maxiter=200, disp=True)
    initial_point = np.zeros(var_form.num_parameters)
    initial_point_red = np.zeros(var_form_red.num_parameters)

    backend = Aer.get_backend("qasm_simulator")

    algo = VQE(qubitOp, var_form, optimizer, initial_point, include_custom=True)
    tic = time.perf_counter()
    res = algo.run(QuantumInstance(backend))
    toc = time.perf_counter()
    print(f'Total run in {(toc - tic)/60.0:0.4f} minutes')
    vqe = res.eigenvalue.real + repulsion_energy
    print('vqe', vqe)

    algo = VQE(qubitOp_red, var_form_red, optimizer, initial_point_red, include_custom=True)
    tic = time.perf_counter()
    res = algo.run(QuantumInstance(backend))
    toc = time.perf_counter()
    print(f'Total run in {toc - tic:0.4f} seconds')
    vqe_red = res.eigenvalue.real + repulsion_energy + energy_shift
    print('vqe_red', vqe_red)
```

Assignees
No one assigned

Labels
No labels

Type
No type

Projects
No projects

Milestone
No milestone

Relationships
None yet

Development
No branches or pull requests

Notifications Customize

Subscribe

You're not receiving notifications from this thread.

Participants

```
backend = Aer.get_backend("qasm_simulator")

algo = VQE(qubitOp, var_form, optimizer, initial_point, include_custom=True)
tic = time.perf_counter()
res = algo.run(QuantumInstance(backend))
toc = time.perf_counter()
print(f'Total run in {(toc - tic)/60.0:0.4f} minutes')
vqe = res.eigenvalue.real + repulsion_energy
print('vqe', vqe)

algo = VQE(qubitOp_red, var_form_red, optimizer, initial_point_red, include_custom=True)
tic = time.perf_counter()
res = algo.run(QuantumInstance(backend))
toc = time.perf_counter()
print(f'Total run in {toc - tic:0.4f} seconds')
vqe_red = res.eigenvalue.real + repulsion_energy + energy_shift
print('vqe_red', vqe_red)
```

The results were:

```
two_qubit_reduction only works with parity qubit mapping but you have jordan_wigner. We switch two_qubit
two_qubit_reduction only works with parity qubit mapping but you have jordan_wigner. We switch two_qubit_
Optimization terminated successfully    (Exit mode 0)
            Current function value: -8.90895214557931
            Iterations: 11
            Function evaluations: 1035
            Gradient evaluations: 11
Total run in 6.8310 minutes
vqe -7.882751995120357
Optimization terminated successfully    (Exit mode 0)
            Current function value: -1.0887940511828818
            Iterations: 7
            Function evaluations: 64
            Gradient evaluations: 7
Total run in 1.1011 seconds
vqe_red -7.881460995424649
```

woodsp-ibm on Jan 26, 2021 Member

The VQE setting, `include_custom=True`, as shown above should be more performant. This initially was the default but we had feedback that qasm_simulator was normally chosen, over statevector, as behavior with shot noise was wanted to be investigated and that it was not the default was problematic as the custom expectation using Aer, as enabled by the above, while its fastest has the same ideal outcome as statevector and people did not expect that.

Since the above should have addressed the original issue I am closing this.

woodsp-ibm closed this as completed on Jan 26, 2021